

12. UI Animation and Icons

Detailed beginner handbook - English and Arabic

Technologies: Rive, Lucide React

What you will learn

Add an accessible icon and understand an animation asset.

Icons should clarify actions; Rive animations should be optional and never distract from important data.

الهدف: بناء مثال صغير وفهمه خطوة بخطوة.

شرح بسيط: تعلم الفكرة أولاً، ثم جرب المثال الصغير ببيانات اصطناعية فقط.

How to study

Study one lesson at a time. Type examples yourself, predict the result, run the code, then change one thing. Keep a notebook of new words, errors, root causes, and fixes.

Course map

Setup and mental model; ten core concepts; a guided mini-project; the real dashboard architecture; debugging; exercises and answers.

Lesson 1 - Setup and mental model

Prerequisites

Windows 10 or 11, VS Code, PowerShell, and permission to install learning dependencies. Use only synthetic data in tutorials and screenshots.

Setup sequence

1. Open PowerShell.
2. Go to learning-examples/12-animation-icons.
3. Read README.md.
4. Run npm install then npm run dev.
5. Read the complete error if it fails.
6. Change one safe value and rerun.

الخطوات: افتح مجلد المثال، شغل الأمر، غير قيمة واحدة، ولاحظ النتيجة.

Mental model

Treat Rive, Lucide React as one layer in a system. Identify the input, the transformation or rule, the output, and possible failures. Understanding that flow is more valuable than memorising syntax.

Checkpoint

Explain: What problem does this technology solve? What is its input? What output should it produce? What can fail?

Lesson 2 - Icons as communication rather than decoration

What is it?

An icon should reinforce a known action, not force users to guess. Pair unfamiliar icons with text and use a consistent visual vocabulary.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In UI Animation and Icons, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
import {Download} from 'lucide-react';  
<button><Download aria-hidden="true"/> Download report</button>
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Icons as communication rather than decoration.

Icons as communication rather than decoration: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 3 - Lucide React imports, sizing, colour, stroke, and labels

What is it?

Lucide icons are React components. Import only needed icons, set size and stroke consistently, inherit colour where possible, and avoid decorative inconsistency.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In UI Animation and Icons, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
<Download size={20} strokeWidth={2} color="currentColor" />  
// Import only the icons that are used.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Lucide React imports, sizing, colour, stroke, and labels.

Lucide React imports, sizing, colour, stroke, and labels: هذا الدرس يشرح: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 4 - Accessible names, tooltips, focus, and keyboard use

What is it?

Icon-only buttons require an accessible name. Keep visible focus, keyboard activation, adequate target size, and a tooltip as supplemental—not primary—information.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In UI Animation and Icons, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
<button aria-label="Close dialog" title="Close">
  <X aria-hidden="true"/>
</button>
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Accessible names, tooltips, focus, and keyboard use.

Accessible names, tooltips, focus, and keyboard use: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 5 - Rive files, artboards, state machines, and animations

What is it?

A .riv file may contain artboards, named animations, and state machines. A state machine transitions according to boolean, number, and trigger inputs.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In UI Animation and Icons, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
// A Rive file can contain an artboard, animations, and a state machine.  
const {RiveComponent}=useRive({src:'/avatar/guardian.riv',stateMachines:'Main'});
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Rive files, artboards, state machines, and animations.

Rive files, artboards, state machines, and animations: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 6 - Rendering Rive with WebGL and fallback behaviour

What is it?

The Rive React renderer loads the asset and renders to WebGL or canvas. Provide dimensions and a fallback when graphics support or the file fails.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In UI Animation and Icons, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
function Avatar(){
  const {RiveComponent}=useRive({src:'/avatar.riv',autoplay:true});
  return <RiveComponent aria-label="Guardian character"/>;
}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Rendering Rive with WebGL and fallback behaviour.

Rendering Rive with WebGL and fallback behaviour: هذا الدرس يشرح: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Inputs, triggers, hover, click, and application state

What is it?

Read state-machine inputs once loaded, update booleans or numbers, and fire triggers for events. Keep application state authoritative rather than hiding logic in animation.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In UI Animation and Icons, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
const inputs=useStateMachineInput(rive, 'Main', 'active');
useEffect(()=>{if(inputs) inputs.value=ready}, [inputs, ready]);
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Inputs, triggers, hover, click, and application state.

Inputs, triggers, hover, click, and application state: اكتب المثال بنفسك، حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 8 - Performance, lazy loading, and resource cleanup

What is it?

Large assets and continuous rendering consume resources. Lazy-load offscreen animation, pause when hidden, limit instances, and release renderer resources on unmount.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In UI Animation and Icons, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
useEffect(()=>()=>rive?.cleanup(),[rive]);  
// Lazy-load the component and pause continuous animation when hidden.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Performance, lazy loading, and resource cleanup.

Performance, lazy loading, and resource cleanup. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 9 - Reduced-motion preferences and optional animation

What is it?

Respect prefers-reduced-motion by pausing nonessential movement or showing a poster. Never make animation required to understand status or complete a task.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In UI Animation and Icons, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
const reduce=matchMedia('(prefers-reduced-motion: reduce)').matches;  
return reduce ?  : <Avatar/>;
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Reduced-motion preferences and optional animation.

Reduced-motion preferences and optional animation: هذا الدرس يشرح: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 10 - Poster images and graceful failure when assets are missing

What is it?

A poster image provides immediate and reliable content while Rive loads. Missing assets should leave a usable interface and a quiet diagnostic message.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In UI Animation and Icons, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
return <picture><source srcSet="/poster.webp"/></picture>;
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Poster images and graceful failure when assets are missing.

Poster images and graceful failure when assets are missing. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 11 - Testing meaningful UI without depending on animation timing

What is it?

Test labels, controls, and state transitions independently of animation timing. A user must be able to complete the workflow when animation is disabled.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In UI Animation and Icons, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
test('download remains usable without animation',()=>{
  render(<Page animations={false}/>);
  expect(screen.getByRole('button',{name:/download/i})).toBeEnabled();
});
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Testing meaningful UI without depending on animation timing.

Testing meaningful UI without depending on animation timing. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Guided mini-project

Project goal

Add an accessible icon and understand an animation asset.

Starting example

```
import {Download} from 'lucide-react';  
<button aria-label="Download report"><Download/> Download</button>
```

Read the code

Find imported tools or page elements. Identify the synthetic input. Find the function, event, route, or transformation that changes it. Finally, locate where the result is displayed, returned, saved, or packaged.

Build sequence

1. Run the untouched example.
2. Record the result.
3. Rename one unclear value.
4. Add a synthetic record.
5. Validate empty input.
6. Validate incorrect input.
7. Add a helpful error.
8. Rerun every case.
9. Compare with exercise-solution.
10. Explain every line.

Definition of done

Normal, empty, and invalid cases behave deliberately; no private data is used; and you can explain every line without reading the guide.

Lesson 8 - The real dashboard

Architecture connection

The application combines Rive, Lucide React with the rest of the stack. The frontend gathers filters and files; the local API validates requests; the data layer transforms workbook rows; charts present aggregates; export tools create files; and the desktop layer packages the application for Windows.

Trace the upload feature

The user selects a workbook. The frontend sends it to the local API. The backend validates the file and sheets. The data layer creates safe tables. The API returns metadata. React updates state. Plotly displays results. Identify exactly where this guide's technology participates.

Privacy and quality

Do not place narrative, contact, or identifying fields in tutorials, logs, screenshots, tests, or exports. Use generated identifiers and synthetic totals. Validate at each boundary and collect only fields required for the calculation.

Architecture exercise

Draw five boxes: UI, API, Data Processing, Export, Desktop Packaging. Add arrows and label the data. Circle the box related to this guide and add two validation checks.

Lesson 9 - Debugging

Reliable debugging loop

1. Reproduce with the smallest input.
2. Read the first meaningful error.
3. Find the exact file and line.
4. Inspect value and type.
5. Compare expected and actual results.
6. Change one thing.
7. Rerun the same case.
8. Add a regression test.

Common beginner mistakes

- Wrong folder or command.
- Missing dependency or inactive environment.
- Misspelled file, variable, field, route, or import.
- Assuming input is always present.
- Mixing setup, processing, and display in one block.
- Hiding an exception rather than understanding it.
- Using real sensitive data in a test.

أخطاء شائعة: تشغيل الأمر من مجلد خاطئ، نسيان تثبيت التبعيات، أو تغيير أشياء كثيرة بمرة واحدة.

Error notebook

Record: command, expected result, actual error, root cause, smallest fix, and test added. This turns errors into reusable knowledge.

Lesson 10 - Exercises and answers

Exercises

1. Explain the technology in three sentences.
2. Recreate the example without copying.
3. Add one normal synthetic record.
4. Handle empty input.
5. Handle invalid input.
6. Improve unclear names.
7. Add or describe one test.
8. Locate the technology in the dashboard.
9. Record one error and root cause.
10. Add one mini-project feature.

تمرين: أضف شهرا أو قيمة أمانة جديدة، ثم اشرح ما حدث بكلماتك.

Answer guide

A strong solution is observable and explainable: the documented command works; output changes for the new record; empty and invalid cases have deliberate results; names communicate purpose; a test states an expected result; and the architecture explanation identifies the correct boundary.

Next step

Next: 13. Desktop Applications with PyWebView